TRAVAUX PRATIQUES

Java - Programmation Orientee Objet

Les Classes Internes

Member Inner Class | Static Nested Class | Local Class | Anonymous Class

Duree totale	1 heure 30 minutes
Nombre d'exercices	4 exercices progressifs
Bareme	100 points
Prerequis	Cours sur les classes internes

Nom : TAJAN Prenom : PIERRE

Groupe : B Date : 13/04/2026 Note : _____ / 100

Introduction et consignes

Objectifs pedagogiques

Ce TP vous permet de mettre en pratique les 4 types de classes internes Java vus en cours.
Chaque exercice cible un type precis, avec une progression en difficulte.
Vous devez implementer le code, expliquer vos choix et repondre aux questions de reflexion.

Consignes generales

- Respectez strictement les noms de classes et de methodes indiques.
- Ajoutez des commentaires Javadoc sur chaque classe et methode creee.
- Compilez et testez votre code avant de passer a l'exercice suivant.
- Les questions de reflexion sont a repondre directement sur ce document.
- Vous pouvez utiliser Java 11 ou superieur.

Tableau des exercices

N.	Sujet	Duree	Difficulte	Points
1	Classe interne membre - Robot et son Bras	20 min	Debutant	20 pts
2	Classe imbriquee statique - Pattern Builder	25 min	Intermediaire	25 pts
3	Classe locale - Filtre de liste	20 min	Intermediaire	20 pts
4	Classe anonyme & Lambda - Evenements	25 min	Avance	35 pts

Exercice 1 - Classe interne membre - Robot et son Bras	20 min	Debutant

Contexte

Vous modelisez un robot industriel. Le robot possede un bras articule dont le comportement depend etroitement de l'etat du robot (allume ou eteint, niveau d'energie). Le bras n'a de sens que dans le contexte du robot : c'est donc un candidat ideal pour une classe interne membre.

Partie A - Implementation (12 points)

Creez une classe Robot contenant une classe interne membre Bras, en respectant les specifications suivantes :

1. La classe Robot doit posseder :
   - Un champ prive String nom
   - Un champ prive boolean allume (false par defaut)
   - Un champ prive int energie (entre 0 et 100)
   - Un constructeur Robot(String nom, int energie)
   - Une methode void allumer() et void eteindre()
   - Une methode Bras getBras() retournant une instance de Bras

2. La classe interne Bras doit posseder :
   - Une methode void saisir(String objet) : affiche un message si le robot est allume et energie > 20, decremente energie de 10
   - Une methode void deposer(String objet) : affiche un message si le robot est allume, decremente energie de 5
   - Une methode void afficherEtat() : affiche le nom du robot, son etat et son niveau d'energie

3. Ecrivez une methode main dans Robot qui cree un robot, tente de saisir avant allumage, puis effectue plusieurs operations.

Zone de code - Exercice 1

```
// Fichier : Robot.java
public class Robot {
    // TODO : declarez les champs


    // TODO : constructeur


    // TODO : methodes allumer() et eteindre()


    // TODO : methode getBras()


    // TODO : classe interne membre Bras
    class Bras {
        // TODO : methodes saisir(), deposer(), afficherEtat()
```

```
    }

    public static void main(String[] args) {
        // TODO : scenario de test



    }
}
```

Partie B - Questions de reflexion (8 points)

Question 1 (3 pts) : Pourquoi la classe interne Bras peut-elle acceder directement aux champs prives de Robot sans getter ? Quel mecanisme Java le permet ?

La classe interne Bras a un accès automatique aux membres privés de sa classe englobante Robot car elle fait partie d'elle. C'est une caractéristique des classes internes non statiques qui induit une poximité entre la classe interne et sa classe externe

Question 2 (2 pts) : Quelle est la syntaxe pour creer une instance de Bras depuis une autre classe que Robot ? Expliquez pourquoi cette syntaxe est differente d'une instanciation classique.

Robot robot = new Robot("RoboX", 50);
Robot.Bras bras = robot.new Bras();
Cette syntaxe est différente car Bras est une inner class non statique : elle est liée à une instance spécifique de Robot. Java nécessite une référence à Robot pour créer l'instance interne.

Question 3 (3 pts) : Si Bras avait aussi un champ int energie, comment distingueriez-vous les deux champs ? Ecrivez l'exemple de code.

```
// Exemple de code :
```

On utilise this.energie pour le champ de Bras et Robot.this.energie pour le champ de Robot :

```
class Bras {
   private int energie = 100;  // champ de Bras

   public void afficherEnergies() {
      System.out.println("Énergie Bras : " + this.energie);     // 100
      System.out.println("Énergie Robot : " + Robot.this.energie); // énergie du robot
   }
}
```

Exercice 2 - Classe imbriquee statique - Pattern Builder	25 min	Intermediaire

Contexte

Le pattern Builder est l'un des usages les plus courants des classes imbriquees statiques en Java. Il permet de construire des objets complexes etape par etape, en evitant les constructeurs a nombreux parametres (telescoping constructor anti-pattern).

> **Rappel : Pourquoi static ?**
> Une classe imbriquee statique est utilisee lorsque la classe n'a pas besoin d'acceder a l'etat (membres d'instance) de la classe externe. Le Builder a besoin d'acceder au constructeur prive de l'objet, mais pas a ses champs pendant la construction.

Partie A - Implementation (15 points)

Creez une classe Ordinateur (immuable - tous les champs sont final) avec une classe imbriquee statique Builder.

4. La classe Ordinateur doit avoir les champs suivants (tous final) :
   - String marque et String processeur (obligatoires)
   - int ramGo (obligatoire, minimum 4 Go)
   - int stockageGo (optionnel, defaut : 256)
   - boolean ssd (optionnel, defaut : true)
   - String carteGraphique (optionnel, defaut : "Integree")
   - double prixEuros (optionnel, defaut : 0.0)

5. La classe Builder (static) doit :
   - Avoir un constructeur Builder(String marque, String processeur, int ram) pour les champs obligatoires
   - Retourner this dans chaque setter pour permettre le chainage (fluent API)
   - Valider les contraintes dans build() : lancer IllegalArgumentException si ram < 4

6. Ecrivez une methode main qui cree 3 ordinateurs differents via le Builder et les affiche.

Zone de code - Exercice 2

```
// Fichier : Ordinateur.java
public final class Ordinateur {
    // TODO : champs final



    // TODO : constructeur prive prenant un Builder
    private Ordinateur(Builder builder) {

    }

    // TODO : getters et toString()
```

```
    // Classe imbriquee statique
    public static class Builder {
        // TODO : champs du Builder


        public Builder(String marque, String processeur, int ram) {
            // TODO
        }

        // TODO : methodes de configuration (retournent this)


        public Ordinateur build() {
            // TODO : validation + return new Ordinateur(this)
        }
    }

    public static void main(String[] args) {
        // TODO : creer 3 ordinateurs via le Builder
    }
}
```

Partie B - Questions de reflexion (10 points)

Question 1 (4 pts) : Expliquez pourquoi le Builder doit etre une classe imbriquee statique et non une classe interne membre. Quelles seraient les consequences si elle n'etait pas static ?

Le builder doit être static car il doit pouvoir être instancié sans avoir besoin d'une instance d'Ordinateur préexistante.C'est son rôle d'en créer une. Si builder n'était pas static, il faudrait d'abord créer un Ordinateur pour pouvoir créer un builder.

Question 2 (3 pts) : Qu'est-ce que le "telescoping constructor anti-pattern" ? Donnez un exemple Java (5+ parametres) qui illustre ce probleme.

```
// Exemple de telescoping constructor (anti-pattern) :
```

Question 3 (3 pts) : Citez 2 exemples de classes imbriquees statiques celebres dans l'API Java standard, et expliquez leur role.

1. 1. Map.Entry<K,V> (dans java.util) est une classe imbriquée statique représentant une paire clé-valeur d'une
2. map. Elle est statique car elle peut exister indépendamment d'une instance de Map.

---

2. AbstractMap.SimpleEntry<K,V> (dans java.util) est une implémentation concrète et statique de Map.Entry, utilisée pour créer des entrées clé-valeur sans lier à une map particulière.

Exercice 3 - Classe locale - Filtre de liste personnalise	20 min	Intermediaire

Contexte

Les classes locales permettent de definir un comportement specifique a l'interieur d'une methode, en capturant les variables locales de cette methode. Vous allez implementer un systeme de filtrage de produits ou les criteres varient selon les parametres de la methode.

Partie A - Implementation (12 points)

7. Creez une classe Produit avec : String nom, String categorie, double prix, int quantite, un constructeur complet et une methode toString().
8. Dans GestionnaireStock, creez la methode filtrerEtTrier(String categorie, double prixMax, int quantiteMin) qui :
   - Definit une interface locale Filtre avec boolean accepter(Produit p)
   - Definit une classe locale FiltreCompose implementant Filtre (teste les 3 criteres)
   - Retourne la liste filtree triee par prix croissant
9. Ecrivez une methode main qui initialise un stock de 8+ produits et appelle filtrerEtTrier avec 2 combinaisons differentes.

Zone de code - Exercice 3

```
// Fichier : GestionnaireStock.java
import java.util.*;

public class GestionnaireStock {
    private List<Produit> stock = new ArrayList<>();

    // TODO : methode pour ajouter un produit

    public List<Produit> filtrerEtTrier(String categorie,
                                        double prixMax,
                                        int quantiteMin) {
        // TODO : definir l'interface locale Filtre


        // TODO : definir la classe locale FiltreCompose




        // TODO : filtrer et trier


        return null; // a remplacer
    }

    public static void main(String[] args) {
        // TODO
    }
}
```

Partie B - Questions de reflexion (8 points)

Question 1 (4 pts) : Que se passerait-il si vous tentiez de modifier les variables categorie, prixMax ou quantiteMin a l'interieur de la classe locale ? Pourquoi Java impose-t-il la contrainte "effectivement finale" ?

Question 2 (4 pts) : Reecrivez la logique de filtrage avec des lambdas et l'API Stream. Quelle version preferez-vous et pourquoi ?

```
// Version lambda + Stream :
public List<Produit> filtrerEtTrierStream(String categorie, double prixMax, int
quantiteMin) {
    return stock.stream()
        // TODO : .filter(...)

        // TODO : .sorted(...)

        // TODO : .collect(...)
        ;
}
```

Exercice 4 - Classe anonyme & Lambda - Systeme d'evenements	25 min	Avance

Contexte

Vous allez implementer un systeme d'evenements (Observer pattern) pour une interface graphique simplifiee. Ce type de systeme est au coeur de Swing, JavaFX et de nombreux frameworks. Il illustre parfaitement l'evolution des classes anonymes vers les lambdas.

> **Objectif double**
> Cet exercice vous fait d'abord utiliser des classes anonymes (comme avant Java 8), puis vous invite a les reecrire avec des lambdas pour comparer les deux approches et comprendre dans quels cas les lambdas ne peuvent pas remplacer les classes anonymes.

Partie A - Implementation avec classes anonymes (20 points)

10. Creez les interfaces fonctionnelles suivantes :
    - ClickListener avec void onClick(int x, int y)
    - KeyListener avec void onKeyPress(char touche)
    - HoverListener avec void onHover(boolean entre)
11. Creez une classe Bouton avec : String label, des listes de listeners, des methodes addXxxListener, des methodes simulerClic(int x, int y), simulerTouche(char c), simulerSurvol(boolean entre).
12. Dans la methode main, creez un bouton "Valider" et enregistrez :
    - 2 ClickListeners anonymes (l'un affiche les coordonnees, l'autre incremente un compteur)
    - 1 KeyListener anonyme qui distingue voyelles et consonnes
    - 1 HoverListener anonyme qui affiche une info-bulle fictive
    - Simulez plusieurs interactions et verifiez que tous les listeners sont appeles

Zone de code - Exercice 4

```
// Fichier : SystemeEvenements.java
import java.util.*;

@FunctionalInterface
interface ClickListener {
    // TODO
}

@FunctionalInterface
interface KeyListener {
    // TODO
}

@FunctionalInterface
interface HoverListener {
    // TODO
}

class Bouton {
    // TODO : champs et listes de listeners
```

```
    // TODO : methodes add*Listener

    // TODO : methodes simuler*

}

public class SystemeEvenements {
    public static void main(String[] args) {
        Bouton btn = new Bouton("Valider");

        // TODO : listeners en classes anonymes


        // TODO : simuler les interactions

    }
}
```

Partie B - Migration vers les lambdas (15 points)

Reecrivez l'integralite de la methode main en remplacant toutes les classes anonymes par des lambdas ou des references de methodes.

```
// Version lambda de la methode main :
public static void mainLambda(String[] args) {
    Bouton btn = new Bouton("Valider");

    // TODO : remplacer toutes les classes anonymes par des lambdas





}
```

Partie C - Questions de reflexion

Question 1 (5 pts) : L'annotation @FunctionalInterface est-elle obligatoire ? Quel est son role exact ? Que se passerait-il si vous ajoutiez une 2e methode abstraite a une telle interface ?

Question 2 (5 pts) : Citez 2 cas ou une classe anonyme reste preferable a un lambda meme en Java 8+. Justifiez avec des exemples de code.

```
// Cas 1 : (classe anonyme obligatoire car...)



// Cas 2 : (classe anonyme obligatoire car...)
```

Question Bonus (5 pts) : Implementez removeListener pour Bouton. Quel probleme rencontrez-vous avec les lambdas lors de la suppression d'un listener, et comment le contournez-vous ?

```
// removeListener + demo + explication du probleme avec les lambdas :
```

Fiche de synthese - A remplir a la fin du TP

Completez ce tableau de synthese une fois tous les exercices termines.

Type de classe interne	Quand l'utiliser ?	Ce que j'ai retenu
Classe interne membre		
Classe imbriquee statique		
Classe locale		
Classe anonyme		

Auto-evaluation

Exercice 1 : ___/20 Exercice 2 : ___/25 Exercice 3 : ___/20 Exercice 4 : ___/35

Ce que j'ai compris :
_____

Ce que je dois retravailler :
_____

Questions pour le prochain cours :
_____